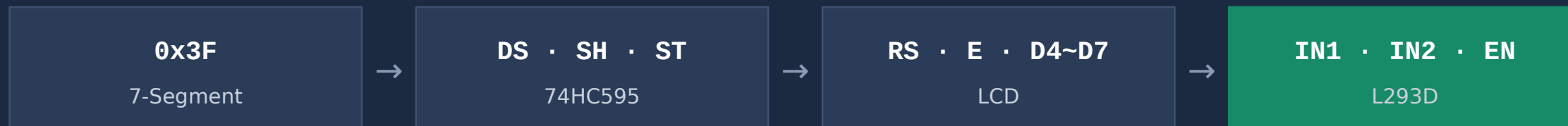


디스플레이 · I2C · 모터

비트맵으로 그리고 , 두 선으로 부르고 , 드라이버로 움직입니다



오늘의 8 교시

오전은 "그리기"(비트맵 · 시프트 · 멀티플렉싱), 오후는 "부르기와 움직이기"(I2C · 모터) — 주인공은 비트 연산

1

7-Segment

비트맵 배열, CC / CA

2

74HC595

DS · SH_CP · ST_CP,
my_shift_out

3

4 자리 멀티플렉싱

타이머 인터럽트, 잔상

4

8×8 매트릭스

행 바이트, 스크롤, set_pixel

5

LCD 1602A

병렬 4 비트, 고정 폭 출력

6

I2C · PCF8591

주소 · 스캐너 · 제어 바이트

7

DC 모터 · 릴레이

H- 브리지, PWM, 안전

8

스텝퍼 · 종합

시퀀스 표, 스마트 환기

오늘의 C — 비트 연산 다섯 개면 된다

Day 1 4 교시 레지스터에서 쓴 연산이 표시 장치 · 통신 · 모터에서 그대로 반복됩니다

연산	의미	오늘 쓰는 곳
<code>(p >> i) & 1</code>	i 번째 비트 꺼내기	7-Segment 출력 , 595 한 비트씩 , 스테퍼 코일
<code>p = mask</code>	그 비트만 켜기	dp 켜기 , 매트릭스 점 켜기
<code>p &= ~mask</code>	그 비트만 끄기	매트릭스 점 끄기
<code>~p</code>	모든 비트 뒤집기	공통 양극 7-Segment, 매트릭스 열 극성
<code>(a << 8) b</code>	두 바이트 이어 붙이기	매트릭스 스크롤 16 비트 띠
<code>0x40 (ch & 0x03)</code>	필드 조합	PCF8591 제어 바이트

모양 · 순서는 코드가 아니라 표 (데이터) 로 : DIGITS[10], img[8], SEQ[8]. 함수는 표를 읽어 비트를 내보내기만 합니다 .

오늘의 부품과 규칙

레일은 Day 2 와 같다 : 위 빨강 = 3V3, 파랑 = GND, 아래 빨강 = 5V (5~8 교시)

부품	핵심	주의
7-Segment (1 · 4 자리)	공통 음극 (CC) / 공통 양극 (CA)	1 교시 첫 예제로 어느 쪽인지 확인
74HC595 · L293D (DIP 16 핀)	반달 홈 = 1 번 핀 쪽	홈을 왼쪽 , 가운데 홈을 건너 꽃기 — 거꾸로면 과열
8×8 매트릭스 (1088AS)	행 8 + 열 8	핀이 행 · 열 순서가 아님 → 핀 표
1602A LCD (병렬 16 핀)	RS · E · D4~D7 (GPIO 6 개)	5V 전원 · RW 는 GND · 대비는 가변저항
PCF8591	ADC 4ch + DAC, 0x48	첫 바이트는 이전 값
L293D + DC 모터	H- 브리지 · EN PWM	모터를 GPIO 에 직접 연결 금지
릴레이 모듈	COM · NO · NC	접점에 220V 연결 금지
ULN2003 + 28BYJ-48	코일 4 개 시퀀스	5V, 멈추면 코일 끄기

라이브러리는 LCD(LiquidCrystal by Arduino, Adafruit) 하나 — I2C 백팩용 LiquidCrystal I2C 가 아닙니다 . 나머지는 직접 구현합니다 .

키트 (KS3014) 부품만으로 — 오늘의 수량 점검

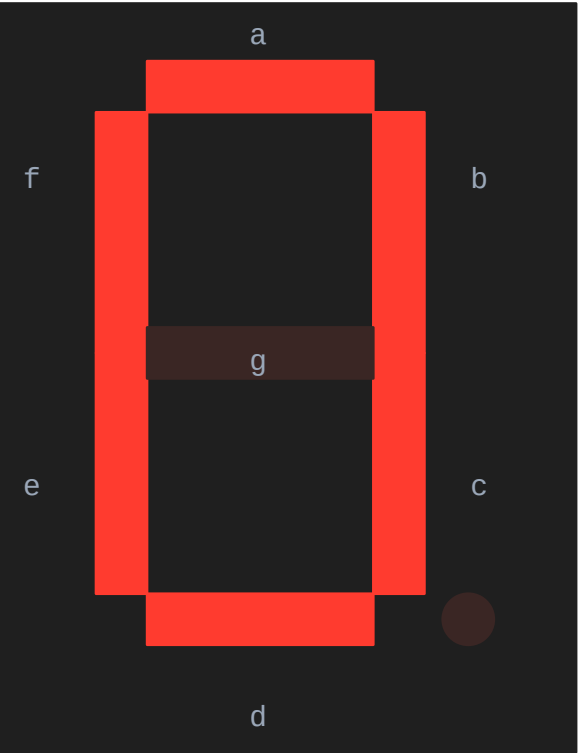
교시가 바뀔 때 쓰지 않는 선은 바로 빼서 다시 쓴다 · 키트에 없는 부품은 쓰지 않는다

부품	키트	오늘 최대	메모
Male-to-Female 선	20	18 (8 교시)	가장 빠듯 — 교시마다 회수
Jumper 선 (M-M)	30	18 (8 교시)	
220Ω 저항	10	8 (3 · 4 교시)	세그먼트 · 매트릭스 열
74HC595N	1	1	여러 개 연결 (Q7') 은 개념만
10kΩ 가변저항	1	1	5 교시부터 LCD 대비 전용
버튼 · LED 빨강 · 초록	4 · 10 · 5	1 · 1 · 1	7 교시 릴레이 실습
Motor + Fan · Water Pump	1 · 1 · 1	1	팬 날개 · 펌프는 7 교시 선택 도전

키트에 없는 것 : 다이오드 · 커패시터 · 외부 전원 → 오늘 쓰지 않음 . 모터 리셋은 천천히 가속 + PC 본체 USB 포트에 연결 . GPIO 확장 보드 · 40 핀 케이블은 Raspberry Pi 용 .

7-Segment — 모양은 1 바이트

a = bit0 ... g = bit6, dp = bit7 — 숫자 모양은 DIGITS[10] 표



숫자	dp g f e d c b a	hex
0	0 0 1 1 1 1 1 1	0x3F
1	0 0 0 0 0 1 1 0	0x06
2	0 1 0 1 1 0 1 1	0x5B
8	0 1 1 1 1 1 1 1	0x7F

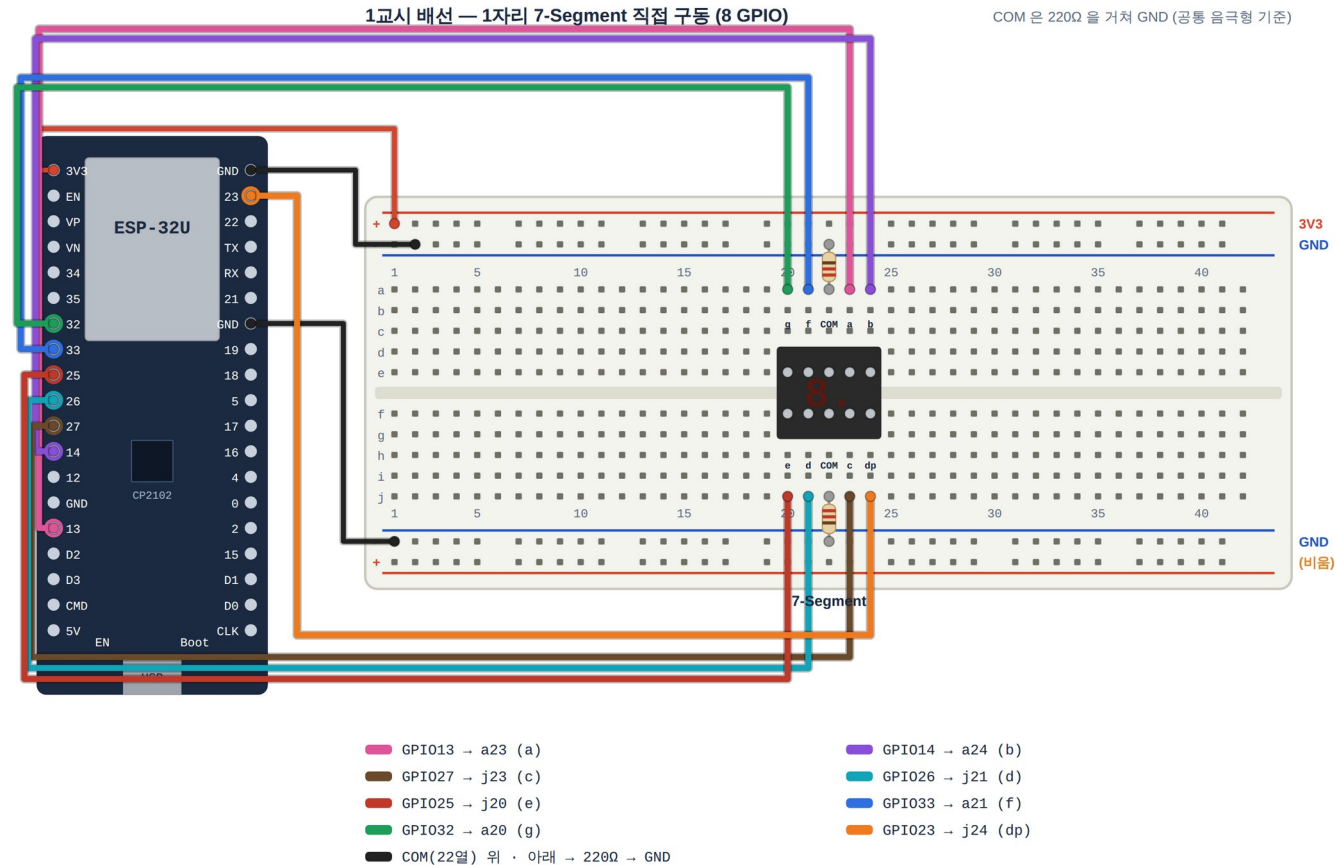
```
void seg_show(uint8_t p) {
    if (COMMON_ANODE) p = ~p;
    for (int i = 0; i < 8; i++)
        digitalWrite(SEG_PINS[i],
                     (p >> i) & 1);
}
```

	COM	켜짐
공통 음극 (CC)	GND (220Ω)	HIGH
공통 양극 (CA)	3V3	LOW

1 교시 배선 — 1 자리 7-Segment 직접 (8 GPIO)

1 교시

a 13 · b 14 · c 27 · d 26 · e 25 · f 33 · g 32 · dp 23 · COM → 220Ω → GND



COM 저항 하나라 "8" 이 "1" 보다 조금 어둡습니다. 아무것도 안 켜지면 CA 부품 — COM 을 3V3 로, COMMON_ANODE = true.

74HC595 — 밀고 (SH_CP) 한 번에 (ST_CP)

2 교시

GPIO 3 개로 출력 8 개 . 래치를 올리기 전에는 출력이 바뀌지 않는다

핀	이름	연결	역할
14	DS	GPIO23	넣을 비트
11	SH_CP	GPIO18	↑ 마다 한 칸 밀기
12	ST_CP	GPIO19	↑ 에 출력으로 복사 (래치)
13 · 10	OE · MR	GND · 3V3	출력 켜기 · 지우기 안 함
15, 1~7	Q0~Q7	a ~ dp	출력
9	Q7'	(비움)	다음 595 의 DS

래치를 안 올림 → 출력 그대로 · MR 떠 있음 → 가끔 지워짐 · OE 떠 있음 → 깜빡임
· 거꾸로 꽂음 → 과열

```
void my_shift_out(uint8_t data) {    // MSB 먼저
    for (int i = 7; i >= 0; i--) {
        digitalWrite(PIN_DS, (data >> i) & 1);
        digitalWrite(PIN_SH, HIGH);
        digitalWrite(PIN_SH, LOW);
    }
}

void write595(uint8_t d) {
    digitalWrite(PIN_ST, LOW);
    my_shift_out(d);        // = shiftOut(DS, SH, MSBFIRST, d)
    digitalWrite(PIN_ST, HIGH);
}
```

🎬 D3-2 시프트 · 래치 — 한 단계씩 , MSB / LSB 비교

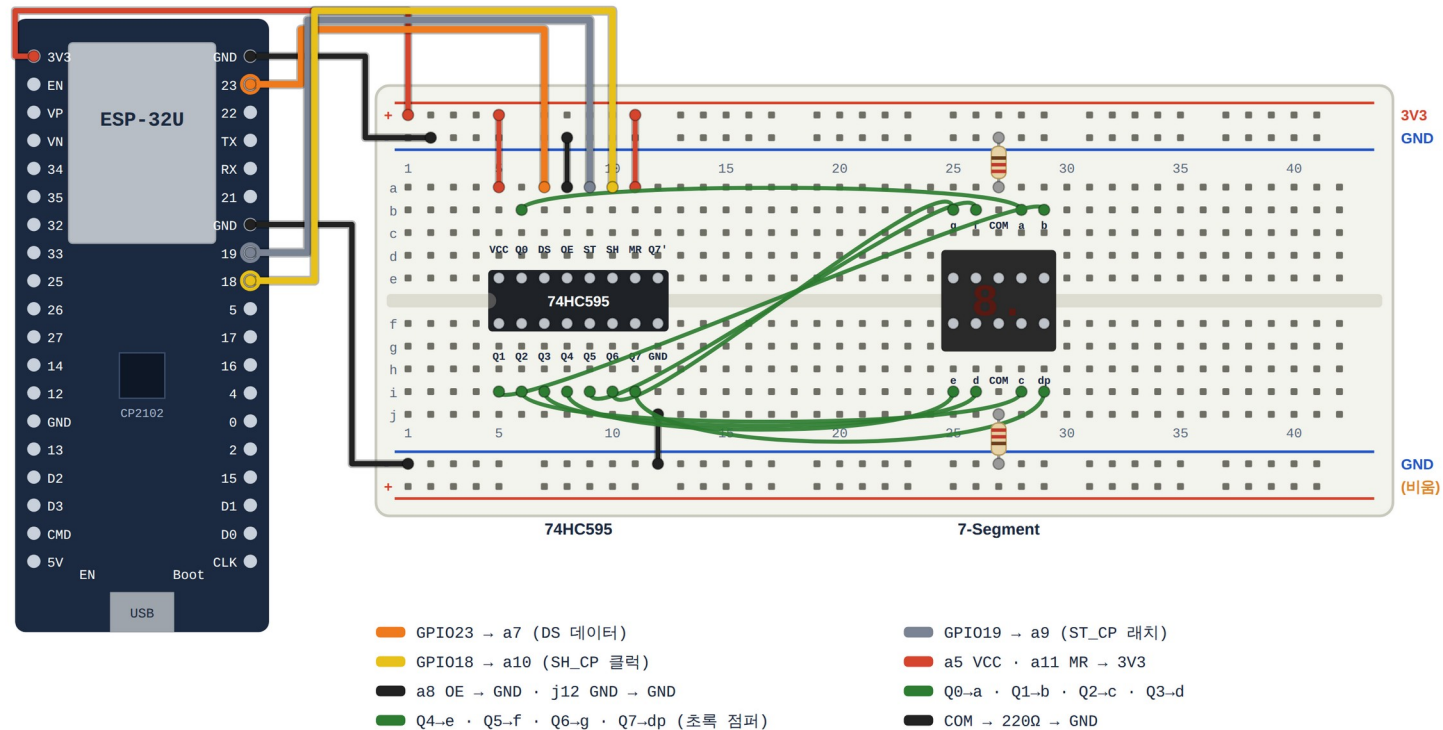
2 교시 배선 — 74HC595 + 1 자리 7-Segment

2 교시

DS 23 · SH_CP 18 · ST_CP 19 · VCC · MR → 3V3 · OE · GND → GND · Q0~Q7 → a~dp (초록 점퍼)

2교시 배선 — 74HC595 로 1자리 7-Segment (3 GPIO)

595 의 홈(반달)이 왼쪽 · Q0~Q7 → a~dp

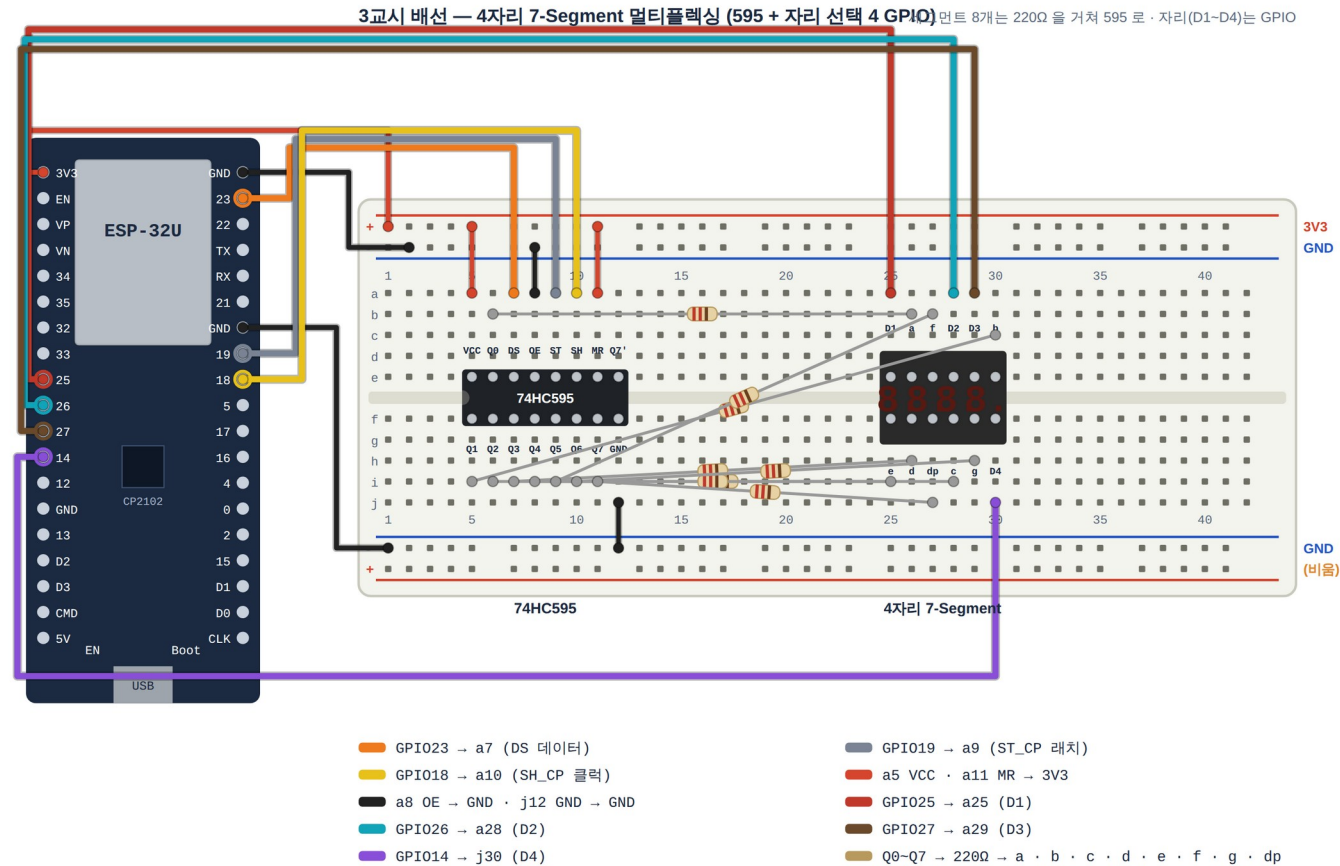


반달 홈이 왼쪽이면 아래 줄 왼쪽부터 1(Q1) ... 8(GND), 위 줄 왼쪽부터 16(VCC) ... 9(Q7').

3 교시 배선 — 4 자리 7-Segment

3 교시

세그먼트 8 개 → 220Ω → 595 · 자리 D1~D4 → GPIO 25 · 26 · 27 · 14

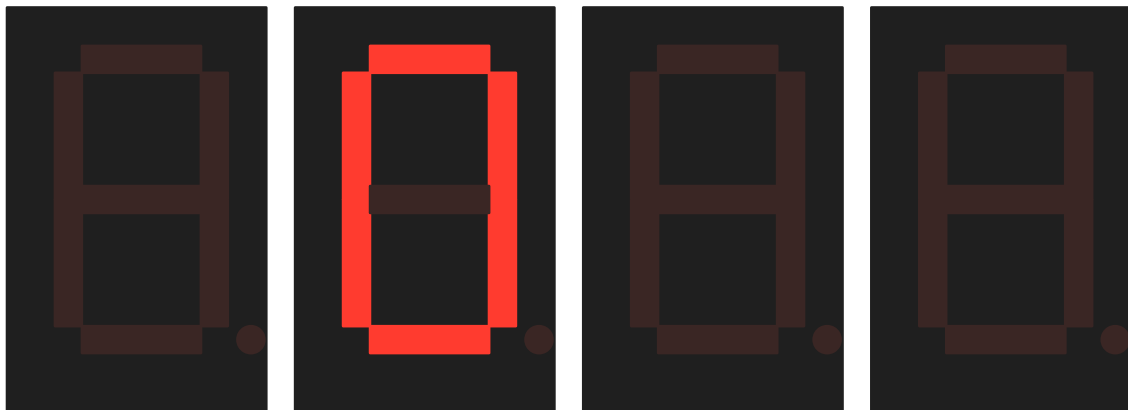


한 자리가 켜질 때 그 D 핀으로 여러 세그먼트 전류가 모입니다 — 한 자리만 켜 둔 채 멈추지 않도록 .

멀티플렉싱 — 한 번에 한 자리, 잔상으로 넷

3 교시

끄기 → 모양 바꾸기 → 켜기를 2ms 마다 . 타이머가 부르므로 loop 가 쉬어도 화면은 돈다



D1

D2

D3

D4

지금 이 순간은 D2 만 켜져 있다 → 2ms 뒤 D3 ... 4 자리 8ms ≈ 125Hz

실수	결과
끄지 않고 바꾸기	고스팅 (옆 자리에 희미하게)
한 자리 20ms	12Hz — 깜빡임
timerBegin(0, 80, true)	2.x 방식 → 컴파일 에러

```
volatile uint8_t disp[4];
volatile int cur = 0;

void IRAM_ATTR onTimer() {
    digitalWrite(DIG[cur], HIGH);    // 끄기
    cur = (cur + 1) % 4;
    write595(disp[cur]);             // 바꾸기
    digitalWrite(DIG[cur], LOW);     // 켜기
}

timer = timerBegin(1000000);        // 1μs
timerAttachInterrupt(timer, &onTimer);
timerAlarm(timer, 2000, true, 0);   // 2ms 반복
```

🎬 D3-3 멀티플렉싱 — 속도 · 고스팅 · loop 가 바쁨

숫자를 자릿수로 — % 10 과 / 10

3 교시

비트 시프트의 10 진수 버전 : % 10 은 맨 아래 자리 , / 10 은 한 자리 밀기

```
void show_number(int n) {  
    for (int k = 3; k >= 0; k--) {          // 오른쪽 자리부터  
        if (n == 0 && k < 3) disp[k] = 0x00; // 앞자리 0 은 빈칸  
        else disp[k] = DIGITS[n % 10];  
        n /= 10;  
    }  
}
```

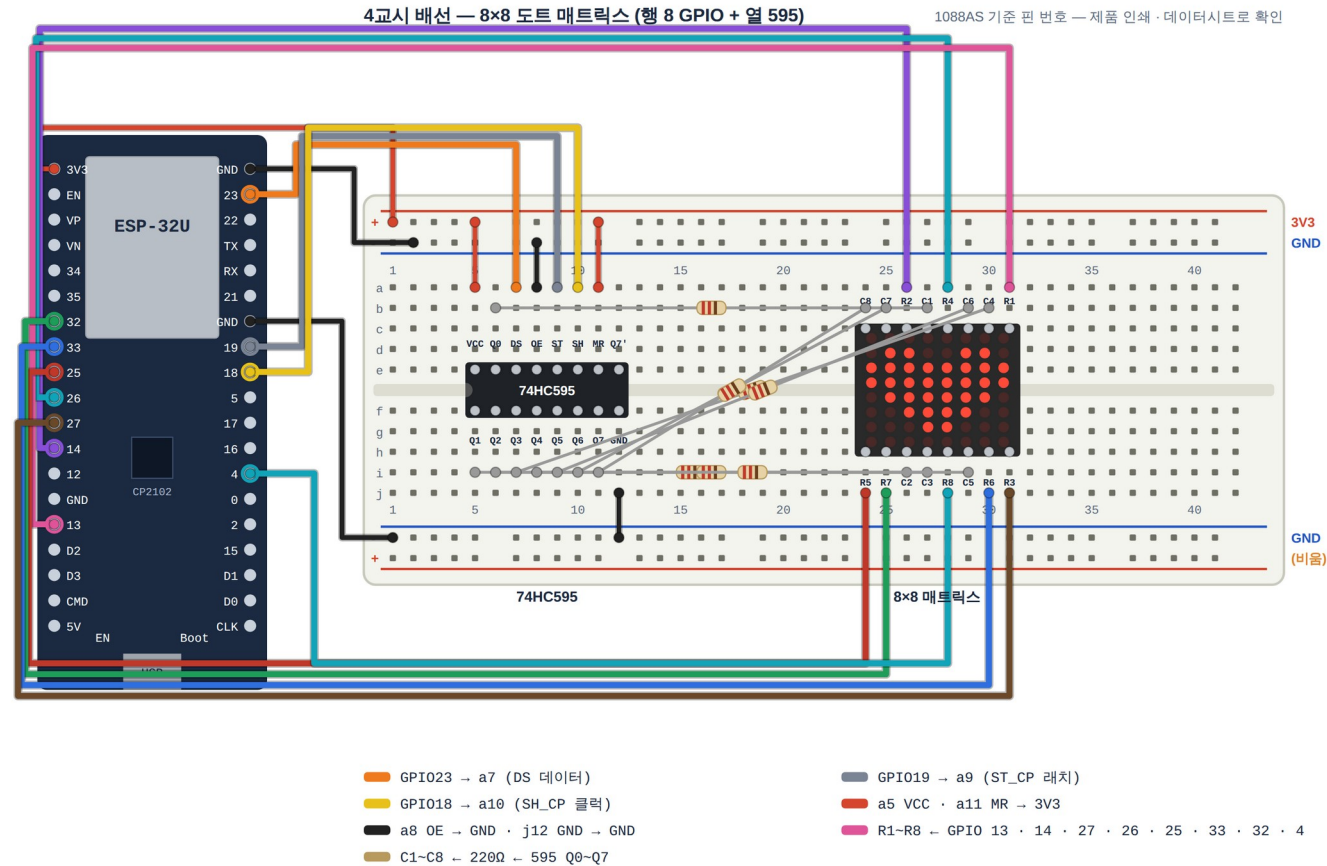
k	n (들어갈 때)	n % 10	disp[k]
3	42	2	"2"
2	4	4	"4"
1	0	—	빈칸
0	0	—	빈칸

k < 3 조건 덕분에 숫자 0 은 맨 오른쪽 한 자리에 "0" 으로 남는다 . loop 는 show_number(count) 만 부르고 , 화면은 타이머가 그린다 — 역할 분리 .

4 교시 배선 — 8×8 도트 매트릭스

4 교시

행 R1~R8 → GPIO 13 · 14 · 27 · 26 · 25 · 33 · 32 · 4 · 열 C1~C8 → 220Ω → 595 Q0~Q7

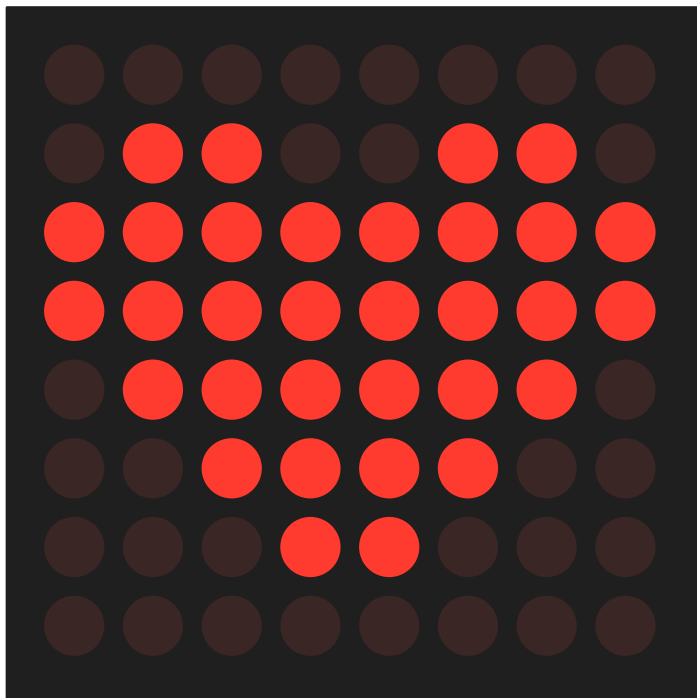


1088AS 핀 : 1 R5 · 2 R7 · 3 C2 · 4 C3 · 5 R8 · 6 C5 · 7 R6 · 8 R3 · 9 R1 · 10 C4 · 11 C6 · 12 R4 · 13 C1 · 14 R2 · 15 C7 · 16 C8. 브레드보드에 안 맞으면 M-F 선으로 .

8 바이트 = 8×8 그림

4 교시

바이트 = 행, bit7 = 왼쪽 열. 행을 한 줄씩 켜는 멀티플렉싱은 3 교시와 같다



```
0b00000000, // 행 0
0b01100110, // 행 1
0b11111111, // 행 2
0b11111111, // 행 3
0b01111110, // 행 4
0b00111100, // 행 5
0b00011000, // 행 6
0b00000000, // 행 7
```

```
// 점 켜기 . 끄기
img[y] |= (0x80 >> x);
img[y] &= ~(0x80 >> x);

// 스크롤 : 16 비트 따의 위 8 비트 창
uint16_t line = (FONT_H[r] << 8)
               | FONT_I[r];
img[r] = (line << shift) >> 8;
```

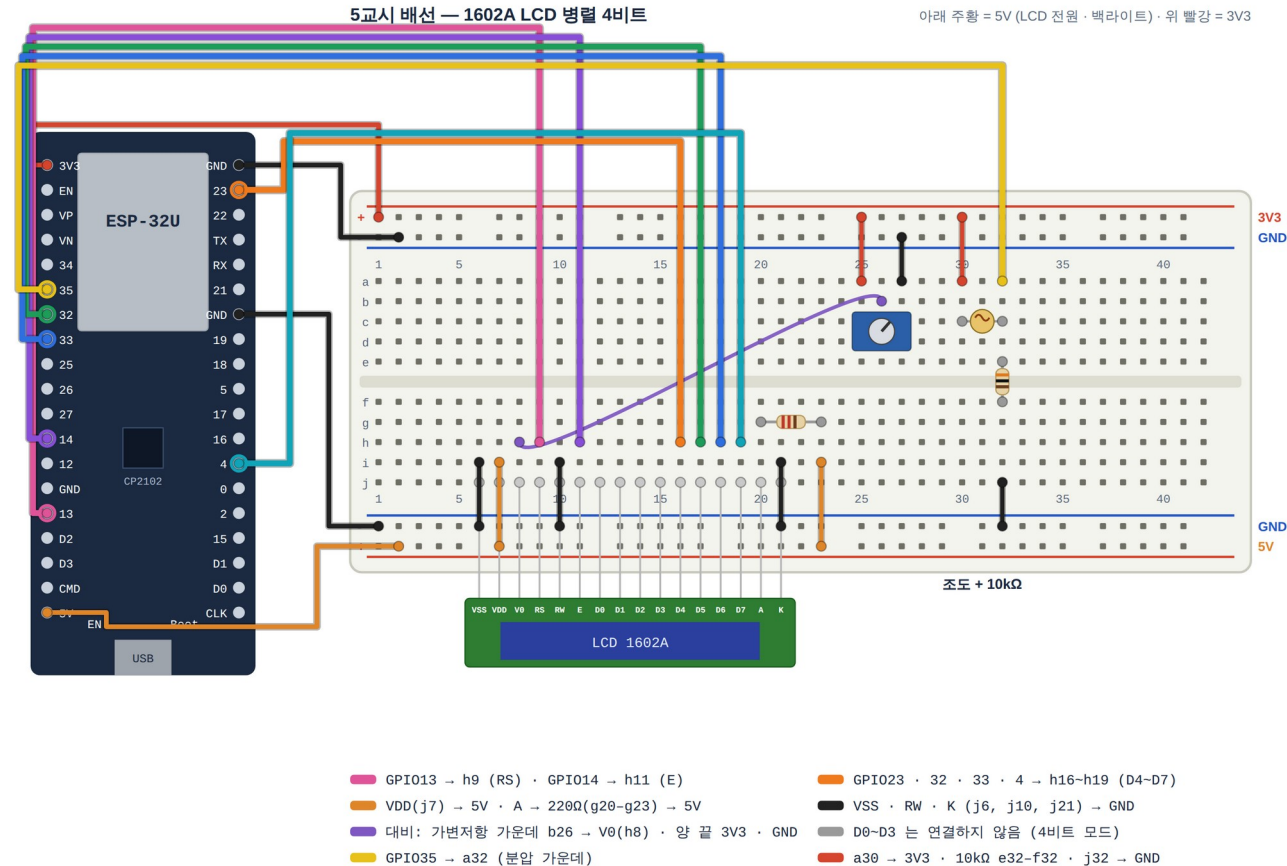
모든 점이 켜지고 그림만 꺼짐 → 극성 반대 (ROW_ON) · 위아래 / 좌우 뒤집힘 → ROWS[] 순서 · 비트 순서 확인

🎬 D3-4 매트릭스 편집기 — 점 그리기 · 스캔 · 스크롤

5 교시 배선 — 1602A LCD 병렬 4 비트

5 교시

RS 13 · E 14 · D4~D7 23 · 32 · 33 · 4 · VDD · A(220Ω) → 5V · VSS · RW · K → GND · V0 ← 가변저항



LCD 16 핀을 j 행 한 줄에 꽂는다 (1 번 VSS 가 왼쪽). D0~D3 은 비워 둔다. 글자가 안 보이면 V0 가변저항을 천천히.

1602A — 16 핀 중 신호선은 6 개

5 교시

한 글자 (8 비트) 를 D4~D7 에 4 비트씩 두 번 — RS 로 명령 / 글자 , E 를 내리는 순간 LCD 가 읽는다

핀	이름	연결	역할
1 · 2	VSS · VDD	GND · 5V	전원 (5V 부품)
3	V0	가변저항 가운데	대비 (글자 진하기)
4	RS	GPIO13	0 = 명령 , 1 = 글자
5	RW	GND	쓰기 전용 — 5V 역류 방지
6	E	GPIO14	HIGH → LOW 에 읽음
7~10	D0~D3	비움	4 비트 모드에서 안 씀
11~14	D4~D7	23 · 32 · 33 · 4	데이터 4 비트
15 · 16	A · K	220Ω→5V · GND	백라이트

```
#include <LiquidCrystal.h>
//           RS  E   D4  D5  D6  D7
LiquidCrystal lcd(13, 14, 23, 32, 33, 4);

lcd.begin(16, 2);
lcd.print("Hello ESP32");
lcd.setCursor(0, 1);    // ( 열 , 행 )
```

3.3V GPIO 로 5V LCD 를 쓸 수 있는 이유 : LCD 는 2.2V 이상을 HIGH 로 읽는다 . RW 를 GND 에 두면 LCD 가 데이터선에 5V 를 내보내지 않는다 .

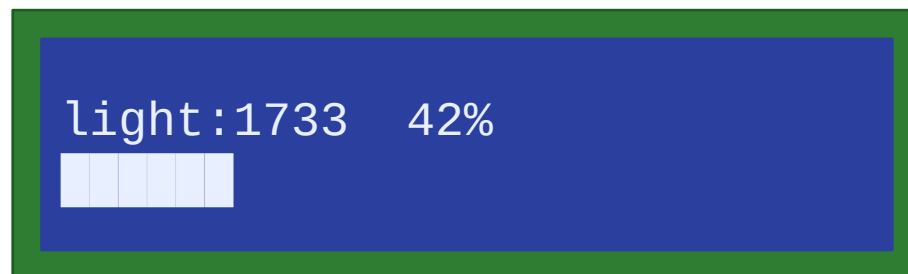
🎬 D3-5 LCD 4 비트 전송 — 글자 보내기 · `setCursor`

LCD 는 덮어쓴 칸만 바뀐다 — 고정 폭으로

5 교시

"1000" 뒤에 "999" 를 쓰면 "9990" — clear() 대신 %4d 로 폭 고정

```
LiquidCrystal lcd(13, 14, 23, 32, 33, 4);  
lcd.begin(16, 2);  
lcd.createChar(0, BAR);           // 사용자 문자 5×8  
  
char line[17];                     // 16 + '\0'  
snprintf(line, sizeof(line), "light:%4d %3d%%", raw, pct);  
lcd.setCursor(0, 0);  
lcd.print(line);
```

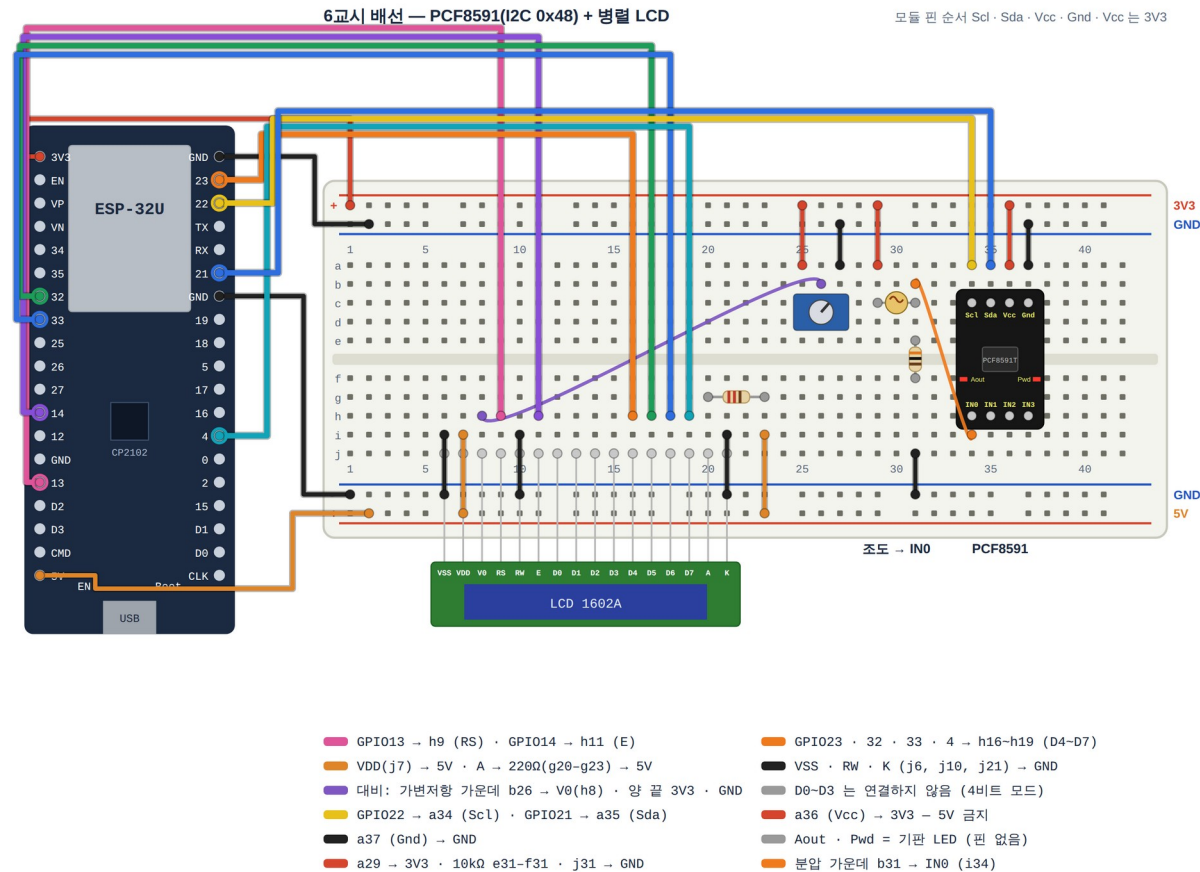


증상	조치
백라이트만 , 글자 없음	V0 가변저항 조정
윗줄 검은 네모 16 개	핀 번호 · begin 확인
깨진 글자	D4~D7 순서 · 느슨한 선
9990 처럼 남음	%4d 고정 폭
VDD 를 3V3 에	매우 흐림 → 5V

6 교시 배선 — PCF8591 (I2C) + LCD

6 교시

모듈 윗줄 Scl 22 · Sda 21 · Vcc 3V3 · Gnd · 아랫줄 IN0 ← 조도 분압 · DAC 는 기판 Aout LED · LCD 는 5 교시 그대로

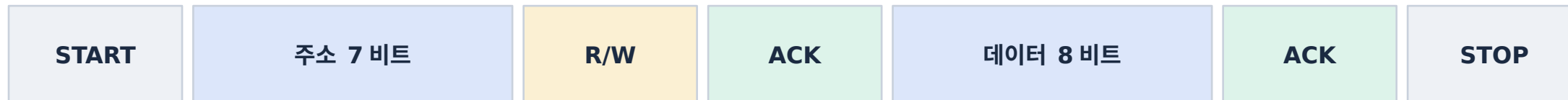


모듈 Vcc 는 반드시 3V3 (기판 풀업이 Vcc 로 연결). 핀 순서는 Scl 이 먼저 . 병렬 LCD 는 I2C 장치가 아니라 스캐너에 나오지 않는 것이 정상 .

I2C — 두 선, 주소로 부른다

6 교시

SCL 이 박자, SDA 에 비트 · 7 비트 주소 + R/W · 장치가 9 번째 박자에 SDA 를 당기면 ACK



```
#include <Wire.h>
Wire.begin(21, 22);

for (uint8_t a = 1; a < 127; a++) {
    Wire.beginTransmission(a);
    if (Wire.endTransmission() == 0)    // 0 = ACK
        Serial.printf(" 발견 : 0x%02X\n", a);
}
```

endTransmission()	의미
0	성공 (ACK)
2	주소 NACK — 장치 없음
3	데이터 NACK
그 밖	버스 오류

아무것도 안 나오면 : SDA · SCL 바뀔, 전원 빠짐, 느슨한 선. 오늘은 0x48(PCF8591) 하나만 나오면 정상.

데이터시트의 제어 바이트를 비트로

6 교시

PCF8591: 8 비트 ADC 4ch + DAC 1ch · 읽은 첫 바이트는 이전 변환 값

7 0	6 DAC	5·4 입력 방식	3 0	2 자동증가	1·0 채널
--------	----------	--------------	--------	-----------	-----------

0x40 = DAC 켜기 + AIN0 · 0x42 = DAC + AIN2

```
int pcf_read(pcf8591_t *p, uint8_t ch) {
    Wire.beginTransaction(p->addr);
    Wire.write(0x40 | (ch & 0x03));
    if (Wire.endTransmission() != 0) {
        p->errors++; return -1;
    }
    Wire.requestFrom(p->addr, (uint8_t)2);
    Wire.read(); // 이전 값 버림
    return Wire.read(); // 0 ~ 255
}
```

```
bool pcf_write_dac(uint8_t v) {
    Wire.beginTransaction(0x48);
    Wire.write(0x40); // DAC 켜기
    Wire.write(v); // 0 ~ 255 → 0 ~ VCC
    return Wire.endTransmission() == 0;
}
```

PWM 은 빠르게 켜고 끄기, DAC 는 진짜 중간 전압. 반환값 (-1 · false) 으로 실패를 알려야 선이 빠진 것을 안다.

pcf8591_t { addr, last[4], errors } — 칩이 두 개 (0x48 · 0x49) 여도 변수만 추가. Day 2 powered_sensor_t 와 같은 설계.

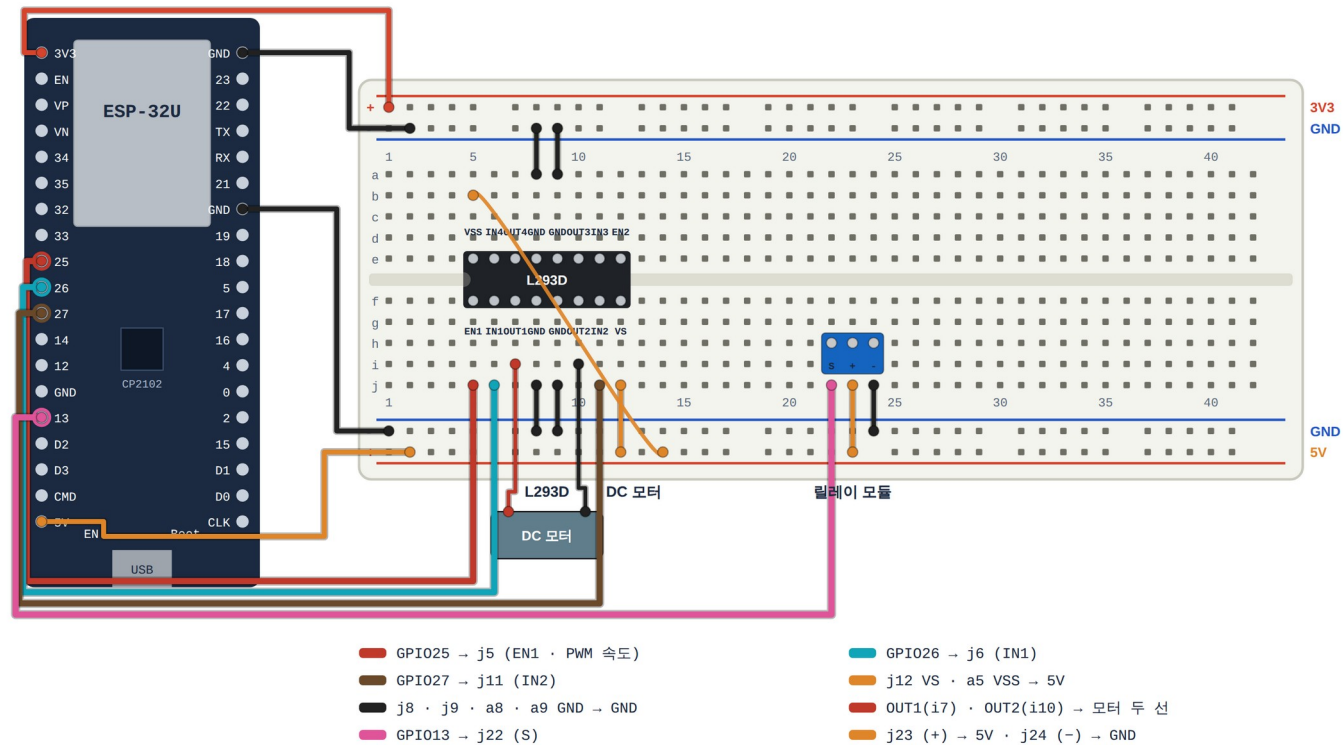
7 교시 배선 — L293D + DC 모터 + 릴레이

7 교시

EN1 ← 25(PWM) · IN1 ← 26 · IN2 ← 27 · OUT1 · OUT2 → 모터 · VS · VSS → 5V · GND 4 개 → GND · 릴레이 S ← 13

7교시 배선 — L293D 로 DC 모터 + 릴레이

아래 주황 = 5V (모터 · L293D · 릴레이 전원)



모터를 GPIO 에 직접 연결 금지 · 릴레이 접점에 220V 금지 · 리셋되면 천천히 가속 + PC 본체 USB 포트 · 모터 축에 키트의 팬 날개 .

H- 브리지 — IN1 · IN2 는 방향 , EN 은 속도

7 교시

스위치 네 개 중 대각선 두 개를 켜면 전류가 모터를 한 방향으로 지나간다

IN1	IN2	모터
HIGH	LOW	정방향
LOW	HIGH	역방향
LOW	LOW	자유 정지
HIGH	HIGH	브레이크

실수	결과
정방향 → 바로 역방향	큰 전류 → 리셋 · 과열 — 멈춘 뒤 반대로
GND 핀 일부만	IC 과열 — 4 개 모두
duty 80 이하	윙 소리만 — 최소 구동값
PWM 1kHz	귀에 들리는 소리

 D3-7 H- 브리지 — IN1 · IN2 · PWM 토글

```
void motor_set(int speed) {          // -255 ~ 255
    if (speed > 255) speed = 255;
    if (speed < -255) speed = -255;
    digitalWrite(PIN_IN1, speed > 0);
    digitalWrite(PIN_IN2, speed < 0);
    ledcWrite(PIN_EN, abs(speed));
}

ledcAttach(PIN_EN, 20000, 8);    // 20kHz — 윙 소리 없음
```

속도를 부호 있는 정수 하나로 : 양수 = 정방향 , 음수 = 역방향 . `ramp_to(&speed, target)` 로 조금씩 — 포인터로 현재 속도 갱신 .

릴레이 — 작은 신호로 큰 스위치 · 4 단계 실습

7 교시

코일 (S · + · -) 은 ESP32 가 , 접점 (COM · NO · NC) 은 별도 회로 — 전기적으로 분리된다

단자	코일 OFF	코일 ON
COM	전원을 넣는 쪽	—
NO	끊김	COM 과 연결
NC	COM 과 연결	끊김

⚠ 이 과정에서 릴레이 접점에는 5V 이하만 . 콘센트 (220V) 절대 금지 .
단자 순서는 모듈 인쇄로 확인 .

앞 단계가 동작해야 다음 단계로 . 한 번에 한 가지만 새로 연결한다 .

① 릴레이 단독
딸깍 · 모듈 LED · ACTIVE_LOW 판별

② 접점 확인
NO 빨강 · NC 초록 LED — 눈으로

③ 모터 켜고 끄기
COM ← 5V · NO → 모터 · 버튼 토글

④ 비상 정지
릴레이 = 주전원 · L293D = 속도

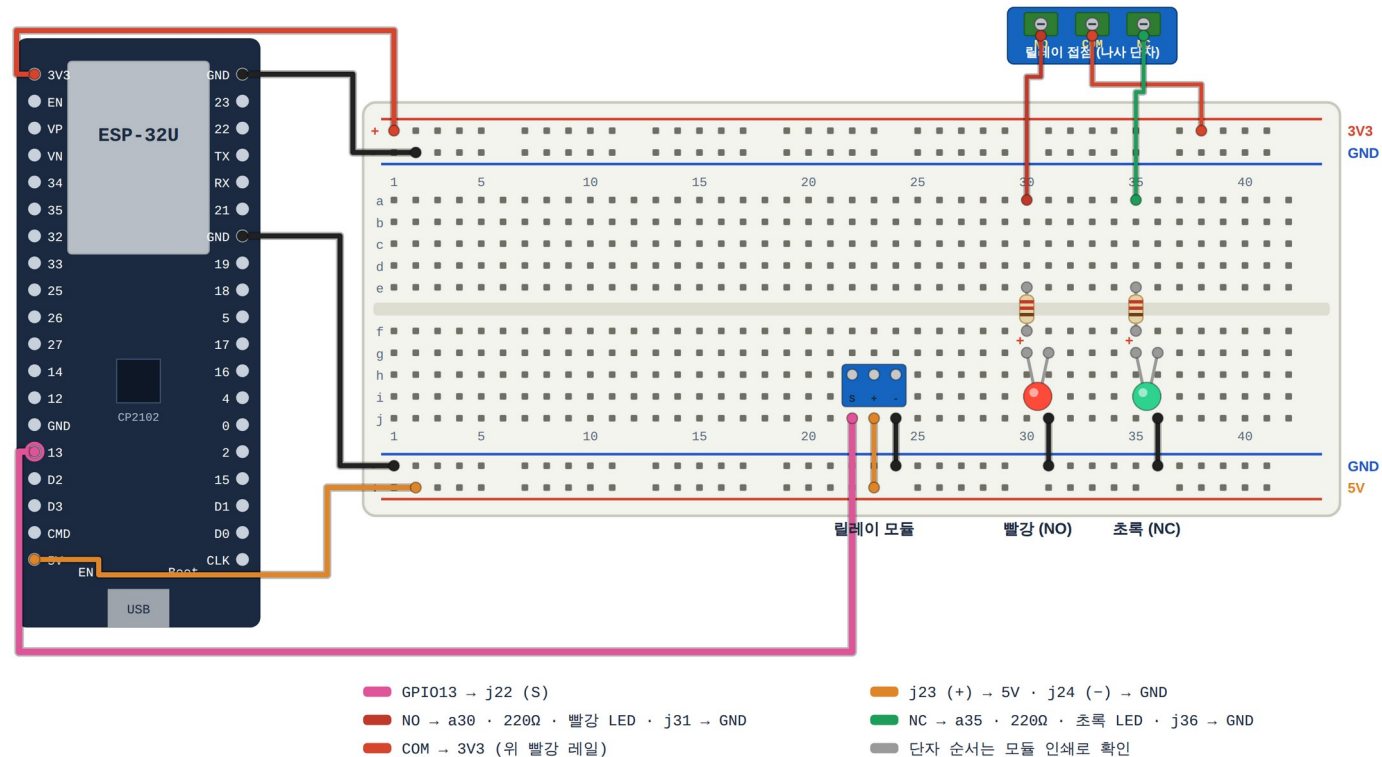
실습 7-3 ② — 점점 확인 : NO 빨강 · NC 초록

7 교시

COM ← 3V3 · NO → 220Ω → 빨강 LED · NC → 220Ω → 초록 LED · 1.5 초마다 코일 ON / OFF

릴레이 ② 배선 — 점점 확인: NO 빨강·NC 초록

COM ← 3V3 · NO → 220Ω → 빨강 LED · NC → 220Ω → 초록 LED



두 LED 가 동시에 켜지는 순간은 없다 — 점점 하나가 NO 와 NC 사이를 오간다 . 반대로 켜지면 NO · NC 를 바꿔 켜는 것 .

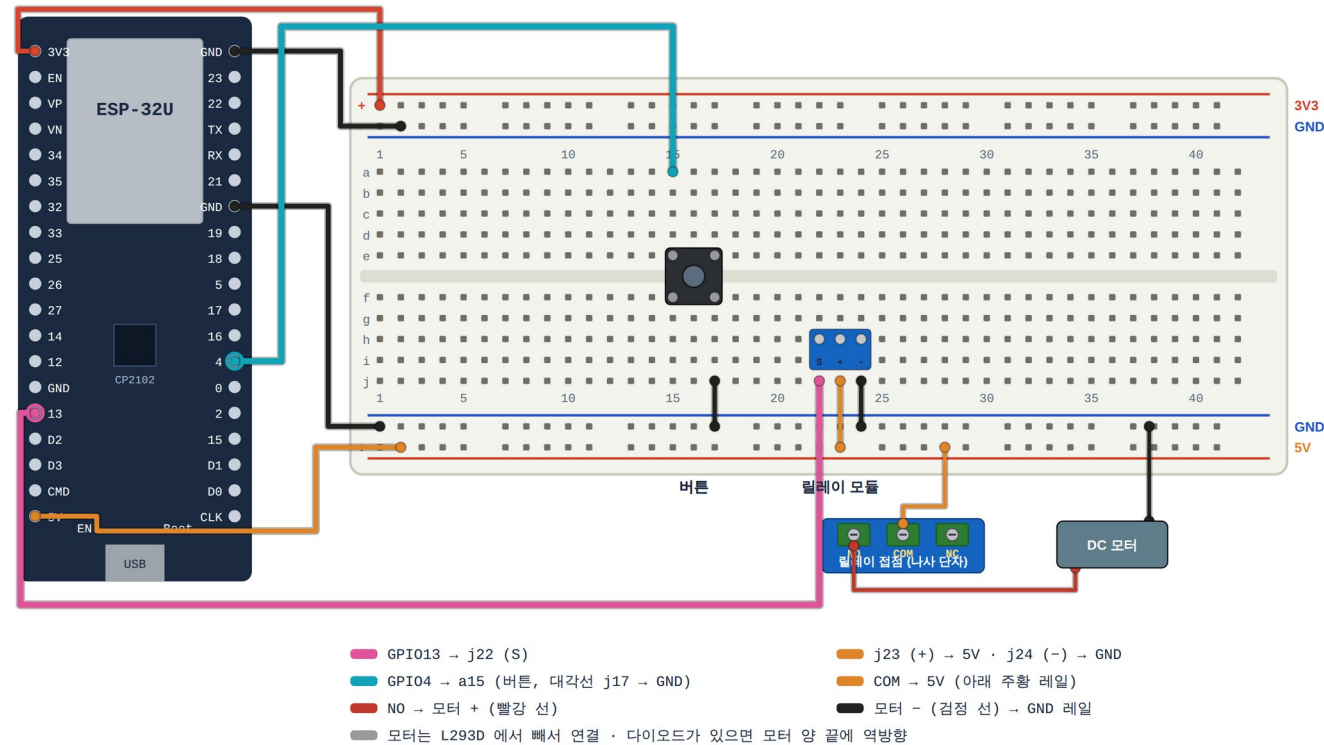
실습 7-3 ③ — 릴레이로 DC 모터 켜고 끄기

7 교시

COM ← 5V · NO → 모터 + · 모터 - → GND · 버튼 GPIO4 로 토글 · 1 초 안의 재전환은 무시

릴레이 ③ 배선 — 릴레이로 DC 모터 켜고 끄기 (버튼 GPIO4)

COM ← 5V · NO → 모터 + · 모터 - → GND



릴레이는 켜기 · 끄기뿐 — 속도 · 방향 조절 불가, 항상 최고 속도. 점점 수명 보호를 위해 MIN_SWITCH_MS.

③ 너무 자주 바꾸지 않게 · ④ 비상 정지 순서

7 교시

접점은 기계 부품 — 전환 횟수만큼 닳는다 · 시작은 무부하, 정지는 주전원부터

```
bool relay_request(bool on) {
    if (on == motor_on) return true;
    if (switch_count > 0 &&
        millis() - last_switch < MIN_SWITCH_MS)
        return false;          // 너무 빠름
    relay_set(on);
    motor_on = on;
    last_switch = millis();
    switch_count++;
    return true;
}
```

```
// 시작 : 속도 0 → 주전원 ON → 천천히 가속
motor_set(0);
relay_set(true);
delay(100);
state = ST_RUN;          // loop 에서 speed += 5

// 비상 정지 : 주전원부터 즉시
relay_set(false);
motor_set(0);
state = ST_STOP;
```

PWM 0 에서 릴레이를 켜면 접점이 전류 없이 붙어 불꽃 (아크) 이 없다 . 코드에 버그가 있어 PWM 이 계속 나와도 주전원이 끊기면 모터는 멈춘다 — 실제 기계의 비상 정지 구조 .

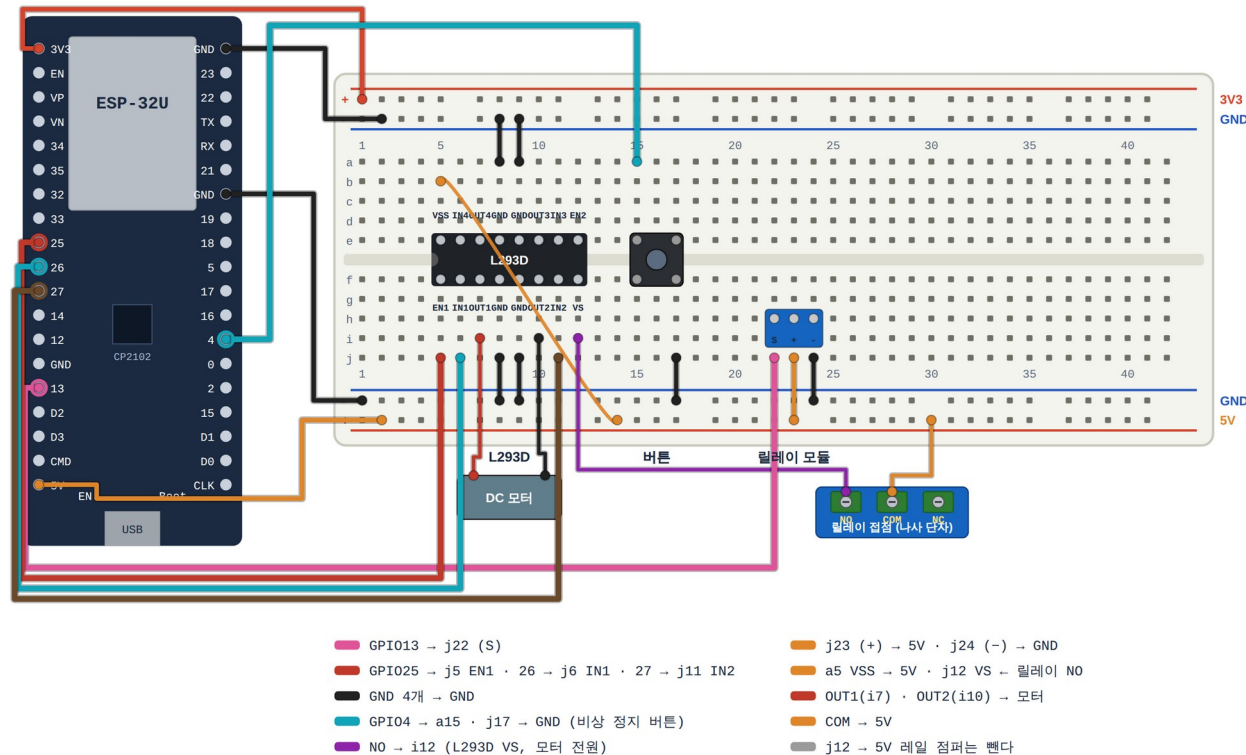
실습 7-3 ④ — 릴레이 = 주전원 , L293D = 속도 · 방향

7 교시

L293D VS(j12) ← 릴레이 NO · COM ← 5V · j12 → 5V 레일 점퍼는 뺀다 · 버튼 = 시작 / 비상 정지

릴레이 ④ 배선 — 릴레이 = 모터 주전원 · L293D = 속도 · 방향

L293D VS(j12) 는 5V 레일이 아니라 릴레이 NO 에서 받는다



릴레이가 꺼지면 L293D 에 모터 전원이 없다 — PWM 과 상관없이 정지 .

L293D 와 릴레이 — 역할이 다르다

속도 · 방향은 반도체 (H- 브리지), 전원 차단 · 분리는 기계 접점 (릴레이)

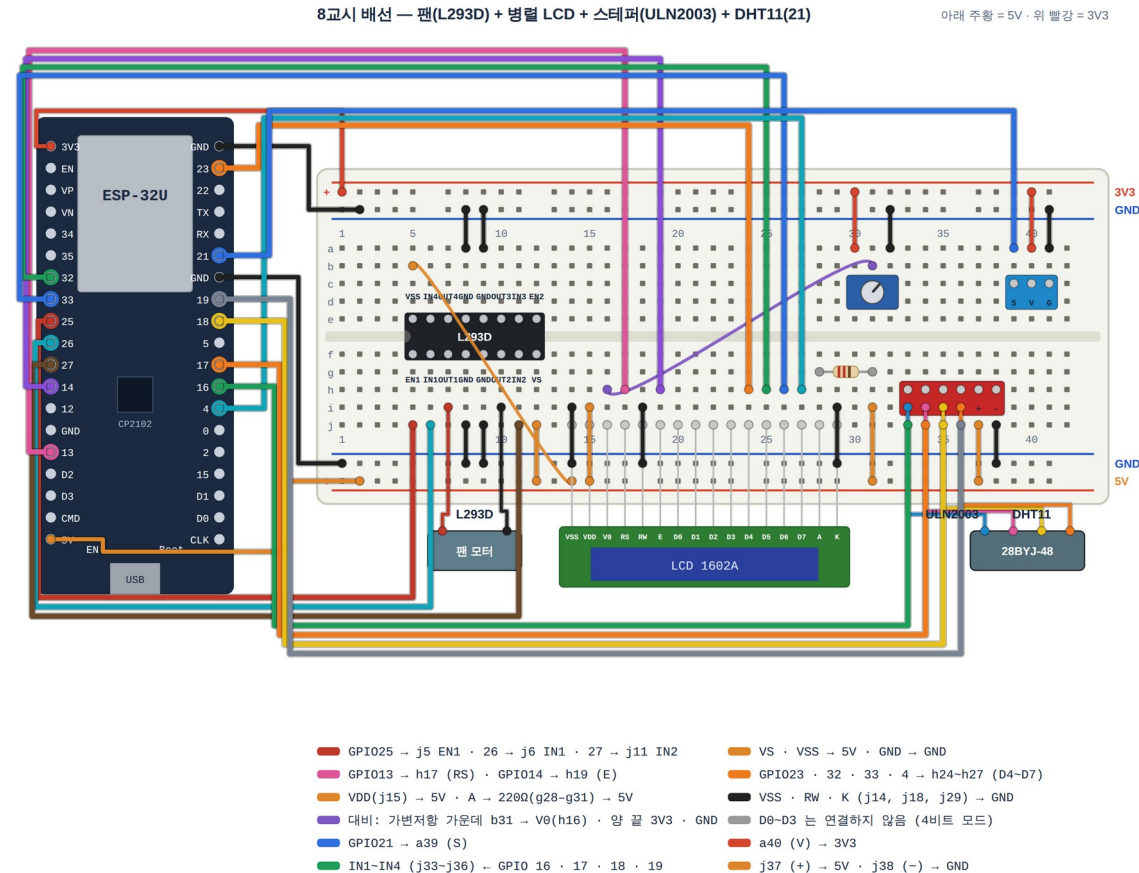
	L293D (H- 브리지)	릴레이
할 수 있는 것	속도 (PWM) · 방향 · 브레이크	켜기 · 끄기만
전환 속도	초당 수만 번 (20kHz)	초당 몇 번 — PWM 금지
수명	반도체 — 사실상 무제한	기계 접점 — 전환 횟수만큼 닳음
ESP32 와 분리	없음 (GND 공통)	있음 (접점은 별도 회로)
소리	없음	딸깍
어울리는 곳	팬 속도 , 바퀴 , 방향 전환	펌프 · 히터 · 램프 전원 , 비상 정지

도전 : ③ 모터 누적 가동 시간 출력 · ④ 버튼을 2 초 이상 눌러야 시작 (실수 방지)

8 교시 배선 — 스테퍼 + 팬 + LCD + DHT11

8 교시

ULN2003 IN1~IN4 ← 16 · 17 · 18 · 19 · 팬 L293D(25 · 26 · 27) · LCD 5 교시 그대로 · DHT11 S ← 21



DHT11 은 LCD D7(GPIO4) 과 겹치지 않게 GPIO21 로 옮겼습니다 . 스테퍼 · 팬 · L293D · LCD 는 5V, DHT11 은 3V3.

스테퍼 — 코일 순서 표를 따라 한 칸씩

8 교시

28BYJ-48 반스텝 약 4096 스텝 = 한 바퀴 · 표를 거꾸로 읽으면 반대로

phase	SEQ	IN4	IN3	IN2	IN1
0	0b0001	0	0	0	1
1	0b0011	0	0	1	1
2	0b0010	0	0	1	0
3	0b0110	0	1	1	0
4	0b0100	0	1	0	0
5	0b1100	1	1	0	0
6	0b1000	1	0	0	0
7	0b1001	1	0	0	1

```
void step_once(int dir) {           // +1 / -1
    phase = (phase + dir + 8) % 8;  // -1 → 7
    for (int i = 0; i < 4; i++)
        digitalWrite(COIL[i], (SEQ[phase] >> i) & 1);
}

void step_motor(int steps, int us) {
    int dir = steps > 0 ? 1 : -1;
    for (int s = 0; s < abs(steps); s++) {
        step_once(dir);
        delayMicroseconds(us);        // 800μs 이상
    }
    for (int i = 0; i < 4; i++) digitalWrite(COIL[i], LOW);
}
```

1 교시 7-Segment 와 같은 " 비트맵 표 " — 이번에는 비트 하나가 코일 하나 . 멈추면 코일을 꺼 발열 방지 .

🎬 D3-8 스테퍼 시퀀스 — 한 스텝 · 방향 · 풀스텝

종합 과제 8-A · 스마트 환기 시스템

DHT11 온도 → 상태 (enum) → 팬 속도 · 환기구 (스테퍼) · LCD — 히스테리시스 27 / 26°C

상태	조건	팬 (L293D)	환기구	LCD 2 행
ST_CLOSED	< 27°C (닫힘은 26 미만에서)	정지	닫힘 0°	VENT:CLOSE FAN: 0
ST_OPEN	≥ 27°C	27°C → 120 ... 32°C → 255	열림 90°	VENT:OPEN FAN:187

TODO	할 일
1	motor_set (정방향)
2	step_motor + 코일 끄기
3	fan_speed 비례식
4	상태 전이 — 바뀔 때만 스텝퍼
5	LCD 고정 폭 출력 (%-5s)

제출 체크리스트

- 27°C 에서 열리고 26°C 아래에서 닫힘
- 열린 상태에서 다시 열리지 않음 (스텝퍼 계속 X)
- 팬 속도가 온도에 따라 변하고 LCD 에 남는 글자 없음
- 상태는 enum, 동작은 함수

확장 : 4 자리 7-Segment 에 온도 24.6 (dp) · 매트릭스에 상태 아이콘 · 35°C 이상 릴레이 경보

오늘의 여섯 문장

- 1 표시 장치의 모양은 비트맵 표로 — $(p \gg i) \& 1$ 로 꺼내고 $|= \cdot \&= \sim$ 로 켜고 끈다
- 2 74HC595 는 DS 에 비트 , SH_CP 로 밀고 , ST_CP 로 한 번에 — 선 3 개로 8 개
- 3 여러 자리 · 여러 행은 멀티플렉싱 . 갱신은 타이머 인터럽트 , 끄기 → 바꾸기 → 켜기
- 4 병렬 LCD 는 RS · E · D4~D7 6 선 (RW 는 GND), I2C 는 두 선에 주소 — 스캐너 · 반환값으로 확인
- 5 데이터시트의 제어 바이트는 비트 연산으로 . 드라이버는 구조체 + 함수 + 오류 반환
- 6 모터는 드라이버를 거쳐 — 방향은 H- 브리지 , 속도는 PWM, 스테퍼는 코일 순서 표

Day 4 예고 FreeRTOS 태스크와 큐 · LittleFS 로그 파일 · Wi-Fi 웹 서버로 스마트폰에서 센서 보기 · 디버깅 · 4 일 종합 프로젝트 · ESP-32U 외부 안테나를 챙겨 오세요 .